

Práctica 0: Sistema de procesamiento de Lenguaje

Compiladores 2027-1 · Facultad de Ciencias, UNAM

Profesora: Ariel Adara Mercado Martínez

Alumno: Santiago Zapata Amezcua

Se trabajó en Ubuntu 24.04 con gcc 13.3.0.

2. Programa en C

Se escribió `programa.c`. Al escribirlo se quitaron los espacios en `<stdio .h>` y en `area (r)`, porque así no compila.

3. `cpp programa.c programa.i`

- a) Se ejecuta el preprocesador. Lee `programa.c`, procesa las directivas y guarda el resultado en `programa.i`.
- b) Al final de `programa.i` está la misma función `main` de `programa.c`, con las mismas instrucciones. Los dos son código C.
- c) Los comentarios se eliminan y las macros se sustituyen: `area (3)` queda como `(3.1416 * 3 * 3)`. Las directivas ya no aparecen.
- d) `programa.i` incluye todo el contenido de `stdio.h` (y `stdlib.h`), por eso es mucho más largo. Ambos tienen las mismas declaraciones de funciones como `printf`, `typedef` y `struct`, pero en `programa.i` ya no hay comentarios ni macros.
- e) Preprocesamiento.

4. `gcc -Wall -S programa.i`

- a) Muestra las advertencias (warnings) más comunes. En este caso no hubo ninguna.
- b) Que solo genere el código ensamblador y no continúe con el ensamblado ni el enlazado.
- c) Se genera `programa.s`, que contiene el programa en lenguaje ensamblador (instrucciones como `movq`, `call`, `ret`).
- d) Compilación.

5. `as programa.s -o programa.o`

- a) Hipótesis: el archivo `.o` tendrá el programa en código máquina y no se podrá leer como texto.
- b) Contiene el código máquina del programa en formato binario (ELF), junto con las cadenas del programa y la información para enlazarlo después. Se ve con símbolos raros porque es binario, no texto; solo se alcanzan a leer las cadenas como `Hola Mundo !`.
- c) El ensamblador de GNU (GNU Assembler).
- d) Ensamblado.

6. Rutas de los archivos

Se buscaron con `find / -name "archivo" 2>/dev/null`.

Archivo	Ruta
ld-linux-x86-64.so.2	/lib64/ld-linux-x86-64.so.2
Scrt1.o	/usr/lib/x86_64-linux-gnu/Scrt1.o
crti.o	/usr/lib/x86_64-linux-gnu/crti.o
crtbeginS.o	/usr/lib/gcc/x86_64-linux-gnu/13/crtbeginS.o
crtendS.o	/usr/lib/gcc/x86_64-linux-gnu/13/crtendS.o
crtn.o	/usr/lib/x86_64-linux-gnu/crtn.o

7. Enlazado con `ld`

a) El comando original marcó que no encontraba `/usr/lib/Scrt1.o` ni `/usr/lib/crti.o`, porque en este sistema están en otra carpeta. La solución fue cambiar las rutas por las del punto 6:

```
ld -o ejecutable -dynamic-linker /lib64/ld-linux-x86-64.so.2 /usr/lib/x86_64-linux-gnu/Scrt1.o /usr/lib/x86_64-linux-gnu/crti.o programa.o -lc /usr/lib/x86_64-linux-gnu/crtn.o
```

b) Con las rutas correctas no mostró errores y se creó el archivo `ejecutable`, que junta `programa.o` con los archivos de arranque y la biblioteca de C (donde está `printf`).

8. Ejecución

```
$ ./ejecutable
Hola Mundo !
Resultado : 28.274401
```

9. Macro `PI` descomentada

a) Se quitó el comentario y se generó `programa_pi.i` con `cpp programa.c programa_pi.i`. Ahora `area (3)` queda como `(3.1415926535897 * 3 * 3)`.

b) Sí cambia un poco. Repitiendo los pasos, el resultado es `28.274334` en lugar de `28.274401`, porque se usa un valor de `PI` más preciso.

10. Segundo programa

`programa2.c` convierte temperaturas de Celsius a Fahrenheit o Kelvin. Se compila con `gcc programa2.c -o programa2` o con `gcc -DESCALA=2 programa2.c -o programa2` para Kelvin.

a) Directivas usadas:

- `#ifndef`: compila un bloque solo si una macro no está definida. Aquí, si no se define `ESCALA` al compilar, se usa 1 (Fahrenheit) por defecto.
- `#if` / `#elif`: compilan código según una condición. Aquí eligen el nombre de la escala y la fórmula de conversión según `ESCALA`.
- `#error`: detiene la compilación con un mensaje. Aquí no deja compilar si `ESCALA` no es 1 ni 2.
- `#undef`: elimina una macro. Aquí se elimina `PASO (10)` para redefinirlo como 25 en la segunda tabla.

Conclusiones

Aunque normalmente se compila con un solo comando de gcc, en realidad participan varios programas: el preprocesador prepara el código, el compilador lo pasa a ensamblador, el ensamblador lo convierte en código máquina y el enlazador lo une con las bibliotecas para crear el ejecutable. También se vio que las directivas del preprocesador pueden cambiar el resultado del programa antes de compilarlo.